

MÉTHODOLOGIE

Ce document explique la démarche derrière ce projet, pas seulement le résultat. Trois principes reviennent régulièrement au fil du développement.

Diagnostiquer par la preuve, jamais par supposition

Chaque bug rencontré a été résolu en observant l'état réel de l'application avant d'agir, jamais en devinant une solution.

CAS RÉEL : Le bouton de refus de permission introuvable

Un test cliquait sur **permission_deny_button**, fonctionnel une première fois puis introuvable une seconde. Plutôt que de supposer un problème de timing, l'écran a été dumpé (uiautomator dump) immédiatement après chaque clic. La comparaison des XML a révélé que l'identifiant du bouton change selon l'historique des refus : **permission_deny_button** au premier refus, **permission_deny_and_dont_ask_again_button** au second. Ce n'était pas un bug, mais un comportement Android natif depuis l'API 23. Le test cible désormais le texte visible plutôt qu'un identifiant instable, et gère les deux refus successifs qu'Android impose.

Cette méthode s'applique à chaque anomalie du projet : le diagnostic du GPU AMD blacklisté par l'émulateur, le crash de snapshot, ou la redirection du checkout vers le login ont tous été confirmés par preuve avant toute correction.

Savoir arrêter, et documenter plutôt que masquer

Le scénario de rotation d'écran programmatique a été tenté selon quatre approches différentes : réglage système dans les deux sens, capability Appium dédiée, version minimale isolée du test. Chacune a échoué avec la même erreur système.

CAS RÉEL : La rotation d'écran

La rotation fonctionne pourtant manuellement, via l'icône de l'interface de l'émulateur. Ça a permis d'isoler la cause réelle : deux mécanismes de rotation distincts existent sur l'émulateur, et seul le mécanisme programmatique échoue sur cette configuration. Le test est conservé dans le code avec un marqueur skip et une raison explicite plutôt que supprimé ou caché.

Un projet où tout passe sans qu'aucune limite ne soit jamais rencontrée est moins crédible qu'un projet qui documente honnêtement ce qui ne fonctionne pas, et pourquoi.

3. Vérifier avant de s'engager, pas après

Avant de choisir l'application cible, sa licence a été vérifiée directement dans le repository plutôt que supposée à partir de sa popularité. Aucun fichier LICENSE n'y est présent, contrairement à d'autres repositories comparables du même éditeur. Ce point est documenté plutôt que passé sous silence, avec la justification du choix malgré cette limite.

Questions posées avant chaque scénario mobile

Un scénario mobile n'a de valeur que s'il répond à une vraie question de comportement, pas seulement s'il déclenche une action à l'écran.

Permissions runtime : *Que se passe-t-il en cas de refus ? L'application redemande-t-elle, combien de fois ? Et en cas d'acceptation, l'accès est-il immédiat ?*

Background / foreground : *L'état d'un formulaire survit-il à une mise en arrière-plan ? Le panier se comporte-t-il différemment d'un simple champ texte ? Un retour rapide diffère-t-il d'un retour après plusieurs cycles ?*

Interruption système : *Une saisie en cours est-elle perdue si un appel interrompt l'utilisateur ? L'application reprend-elle normalement après ?*

Rotation d'écran : *L'interface reste-t-elle utilisable après un changement d'orientation ? Un champ rempli avant rotation est-il toujours accessible après ?*

Ces questions, plus que le code lui-même, définissent ce qui distingue un test mobile pertinent d'un test qui se contente d'exécuter des actions sans rien valider de réel.